

Neighbourhood-Based Collaborative Filtering

Alberto Paccanaro

EMap – FGV

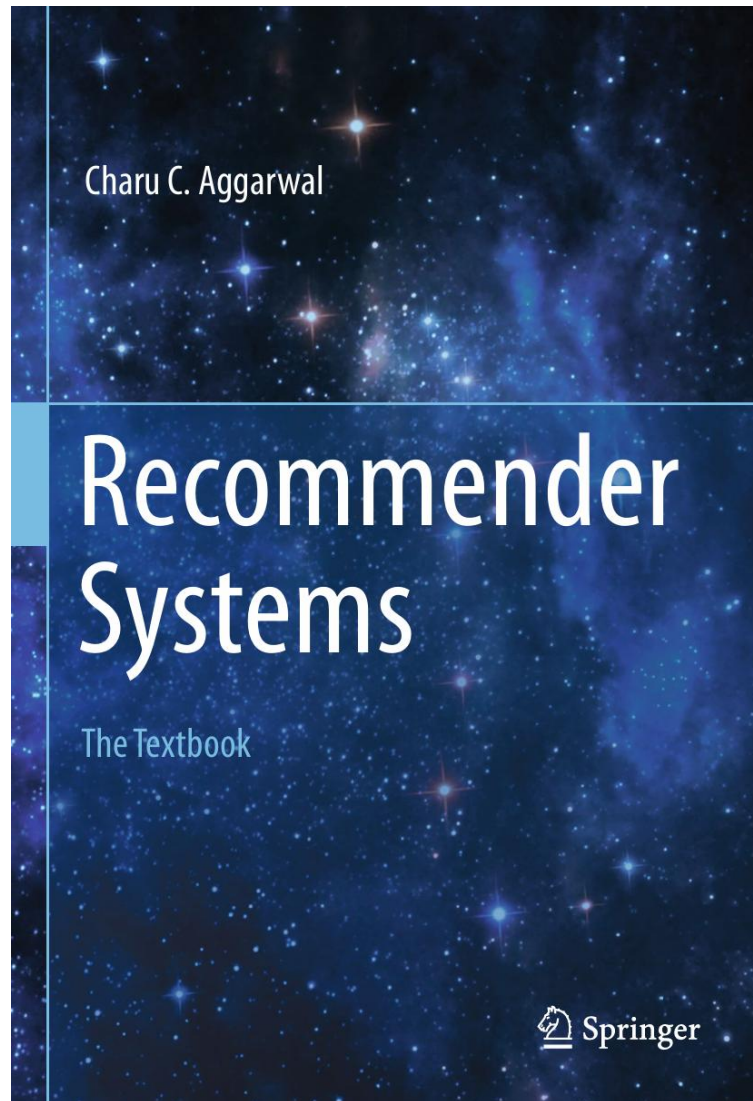
<https://paccanarolab.org>

O material e as imagens nestes slides são de (ou adaptadas de):

Charu C. Aggarwal, *Recommender Systems – The Textbook*, Springer, 2016 – Chapter 2

X.Ning, G.Karypis *SLIM: Sparse Linear Methods for Top-N Recommender Systems*, ICDM 2011

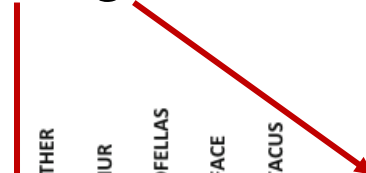
Textbook



The problem

To recommend items to users

- Different type dataset gives rise to different sets of techniques
- An important case is when I have past user activities/ preferences (**ratings**)



	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U ₁	1			5		2
U ₂		5			4	
U ₃	5	3		1		
U ₄			3			4
U ₅				3	5	
U ₆	5		4			

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U ₁	1			1		1
U ₂		1			1	
U ₃	1	1		1		
U ₄			1			1
U ₅				1	1	
U ₆	1		1			

(a) Ordered ratings

(b) Unary ratings

FACTS:

- similar users display similar patterns of rating behaviour
- similar items receive similar ratings

If past Rating Data is available, I can exploit the above facts

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U ₁	1			5		2
U ₂		5			4	
U ₃	5	3		1		
U ₄			3			4
U ₅				3	5	
U ₆	5		4			

(a) Ordered ratings

	GLADIATOR	GODFATHER	BEN-HUR	GOODFELLAS	SCARFACE	SPARTACUS
U ₁	1			1		1
U ₂		1			1	
U ₃	1	1		1		
U ₄			1			1
U ₅				1	1	
U ₆	1		1			

(b) Unary ratings

Collaborative Filtering: ratings predicted using

- 1) **Neighbourhood-Based**
- 2) **Model-Based**

Let us begin by looking at the data...

The data

		items									
users		1	0	0	0	2	0	3	0	4	0
		4	0	0	3	0	4	0	0	0	1
		0	0	4	0	0	0	1	0	3	0
		5	0	0	0	0	5	0	1	0	0
		0	0	4	0	3	0	2	0	0	0

1) It's very sparse!

Datasets	rows	columns	density
Movielens	943	1682	6.3%
Drug-SE	759	994	5%
ModCloth	5419	32089	0.05%

2) The meaning of the “0” entries

- “missing values” zeros that should be turned into ratings
- “real 0 ” zeros that should stay zero

3) Long tail distribution!

- more difficult to provide robust rating predictions in the long tail.
- ratings of these high frequency items are not representative of the low-frequency items

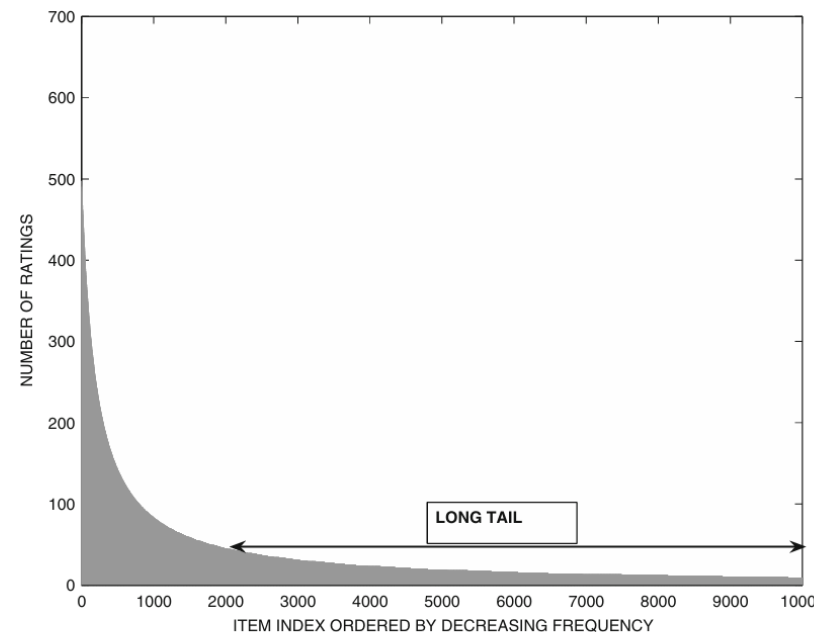


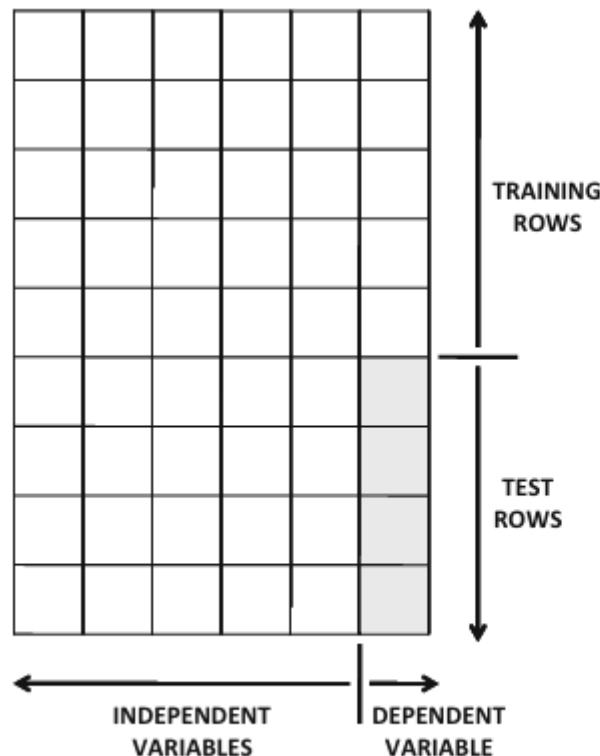
Figure 2.1: The long tail of rating frequencies

Many recommendation algorithms have a tendency to suggest popular items rather than infrequent items ☹

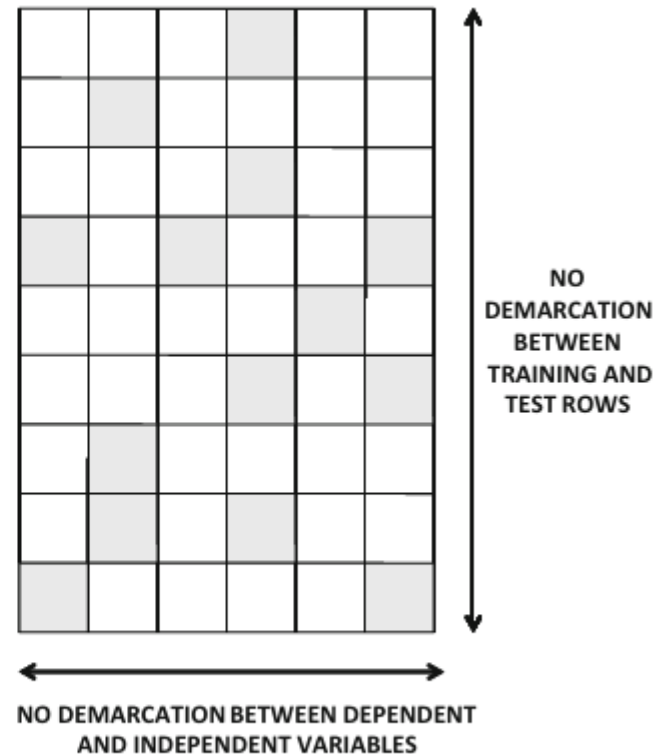
IMPORTANT: the high-frequency items tend to be relatively competitive items with little profit. Most of the profit comes from selling items in the long tail.

Similar to standard classification problem, but different

Careful!



(a) Classification



(b) Collaborative filtering

Technical goals of RecSys

- **Predicting rating values.**
 - Predict the rating, e.g. 1-5 by learning from past rating values.
 - *Matrix completion problem.*
- **Ranking items for users.**
 - Rating values are not important.
 - Top-k users to target for a particular item.
 - Top-k items to suggest to a particular user.
 - *Top-k recommender system.*

Other goals of RecSys

- **Relevance:** recommend items that the user likes.
- **Novelty:** items that the user have not seen before.
- **Serendipity:** unexpected/surprising items are recommended.
- **Diversity:** recommend items that are different.

Neighbourhood-Based Collaborative Filtering

The problem

n items

	5	?	2	4
	?	5	?	?
	?	3	1	?
	4	?	?	3
	?	2	3	?
	1	3	?	1

m user

R

- user-item ratings matrix R , incomplete, $m \times n$ matrix R , with m users and n items.
- Only a small subset of the ratings matrix is observed.

The idea of collaborative filtering

User-based : *Similar users have similar ratings on the same item.*

- The ratings provided by users that are similar to a target user A are used to make recommendations for A.
- The predicted ratings of A are computed as the weighted average values of these “peer group” ratings.
 - ❑ ratings are predicted using the ratings of neighbouring users
 - ❑ neighbourhoods are defined by similarities among users (rows of R)

Item-based: *Similar items are rated in a similar way by the same user.*

- Determine a set S of items, which are most similar to target item B.
- Extract the ratings in S, which were given by A.
- The weighted average of these ratings is used to compute the predicted rating of user A for item B.
 - ❑ ratings are predicted using the user's own ratings on similar items
 - ❑ neighbourhoods are defined by similarities among items (columns of R)

1) User-based collaborative filtering

(A) The Similarity computation is tricky...

different users may have rated different sets of items

I_u = set of item indices for which ratings have been specified by user (row) u .

Pearson correlation coefficient

$$\begin{aligned}\text{Sim}(u, v) = \text{Pearson}(u, v) &= \mu_u = \frac{\sum_{k \in I_u} r_{uk}}{|I_u|} \\ &= \frac{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u) \cdot (r_{vk} - \mu_v)}{\sqrt{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u)^2} \cdot \sqrt{\sum_{k \in I_u \cap I_v} (r_{vk} - \mu_v)^2}}\end{aligned}$$

Cosine

$$\text{RawCosine}(u, v) = \frac{\sum_{k \in I_u \cap I_v} r_{uk} \cdot r_{vk}}{\sqrt{\sum_{k \in I_u \cap I_v} r_{uk}^2} \cdot \sqrt{\sum_{k \in I_u \cap I_v} r_{vk}^2}}$$

Note:

- 1) calculated on the set of item indices for which both user u and v have specified ratings
- 2) Strictly speaking, μ_u and μ_v should be computed only over the items that are rated both by users u and v .

(B) Choosing the peer group of the target user is tricky...

the number of observed ratings in the top- k peer group of a target user may vary significantly depending on the item

- closest k users are found for the target user separately for each predicted item, such that each of these k users have specified ratings for that item.
- The weighted average of these ratings can be returned as the predicted rating for that item.

(C) Users scaling

different users may provide ratings on different scales

- Raw ratings, need to be mean-centered row-wise

$$s_{uj} = r_{uj} - \mu_u$$

- the weighted average of the mean-centred rating of an item in the top- k peer group of target user u is used to provide a mean-centred prediction.
- The mean rating of the target user must then be added back to this prediction

Putting it all together

$P_u(j)$ = set of k closest users to target user u , who have ratings for item j .

$$\begin{aligned}\hat{r}_{uj} &= \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot s_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|} = \\ &= \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot (r_{vj} - \mu_v)}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|}\end{aligned}$$


- Go through the example in section 2.3.1 !

2) Item-based collaborative filtering

- peer groups are constructed in terms of items
- similarities need to be computed between items (columns)
- As before, each row of the ratings matrix is centred to a mean of zero.

U_i = set of user indices who have ratings for item i .

$Q_t(u)$ = the top-k matching items to item t , for which the user U has ratings

$$\text{AdjustedCosine}(i, j) = \frac{\sum_{u \in U_i \cap U_j} s_{ui} \cdot s_{uj}}{\sqrt{\sum_{u \in U_i \cap U_j} s_{ui}^2} \cdot \sqrt{\sum_{u \in U_i \cap U_j} s_{uj}^2}}$$

$$\hat{r}_{ut} = \frac{\sum_{j \in Q_t(u)} \text{AdjustedCosine}(j, t) \cdot r_{uj}}{\sum_{j \in Q_t(u)} |\text{AdjustedCosine}(j, t)|}$$

“adjusted” because the ratings are mean centred

We leverage the user’s own ratings on similar items in the final step of the prediction.

- Go through the example in section 2.3.2 !

Points on Collaborative Filtering

- Item-based methods often provide more relevant recommendations because: similar items are identified to a target item, and *the user's own ratings on those items are used* to extrapolate the ratings of the target.
- **Interpretability**: easy to justify why a specific item is recommended (especially item-based).
- Recommendations are relatively stable with the addition of new items and users.
- Main disadvantage: **limited coverage** because of data sparsity.

A Regression Modelling View of Neighbourhood Methods

CF predict ratings as linear combination of either the ratings of the same item by neighbouring users, or of the same user on neighbouring items.

$$\hat{r}_{uj} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot (r_{vj} - \mu_v)}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|}$$

- **It is very similar to a linear regression model.** But the coefficients of the linear function are chosen in a heuristic way with the user-user similarities, rather than with the use of an optimization model.

Can we do better by learning the weights?

User-Based Nearest Neighbour Regression

Model:

$$\hat{r}_{uj} = \mu_u + \sum_{v \in P_u(j)} w_{vu}^{user} \cdot (r_{vj} - \mu_v)$$

controls the portion of the predictions for user u , which comes from the similarity to user v

- $P_u(j)$ = set of k closest users to target user u , who have ratings for item j (can use Pearson correlation).
- $P_u(j)$ is defined by:
 1. determining the k closest peers for each user;
 2. retaining those for which ratings are observed ($\rightarrow k$ needs to be set to much larger values than before...)
- w_{vu}^{user} is only defined for the k different values of v (user indices) that are closest to user u on the basis of the Pearson coefficient ($\rightarrow$ less regression coefficients)

Learning

Use the observed ratings to set up a least-squares optimization problem over the unknown values of w_{vu}^{user} to minimize the error.

- The optimization problem is set up for each target user u .

$$\begin{aligned} \text{Minimize } J_u &= \sum_{j \in I_u} (r_{uj} - \hat{r}_{uj})^2 \\ &= \sum_{j \in I_u} \left(r_{uj} - \left[\mu_u + \sum_{v \in P_u(j)} w_{vu}^{user} \cdot (r_{vj} - \mu_v) \right] \right)^2 \end{aligned}$$

I_u = set of item
rated by user u .

- Over all users it becomes:

$$\text{Minimize } \sum_{u=1}^m J_u = \sum_{u=1}^m \sum_{j \in I_u} \left(r_{uj} - \left[\mu_u + \sum_{v \in P_u(j)} w_{vu}^{user} \cdot (r_{vj} - \mu_v) \right] \right)^2$$

- A regularization term should be included

Item-Based Nearest Neighbour Regression

- The regression learns and leverages item-item correlations (rather than user-user correlations.)

Model:

$$\hat{r}_{ut} = \sum_{j \in Q_t(u)} w_{jt}^{item} \cdot r_{uj}$$

controls the portion of the predictions for item t , which comes from its similarity to item j

- $Q_t(u)$ = subset of the k nearest neighbours of the target item t (can use Adjusted Cosine)

Learning

Use the observed ratings to set up a least-squares optimization problem over the unknown values of w_{jt}^{item} to minimize the error.

- the optimization problem is set up for each target item t .

$$\begin{aligned}\text{Minimize } J_t &= \sum_{u \in U_t} (r_{ut} - \hat{r}_{ut})^2 \\ &= \sum_{u \in U_t} (r_{ut} - \sum_{j \in Q_t(u)} w_{jt}^{item} \cdot r_{uj})^2\end{aligned}$$

U_i = set of users
who rated item i .

- Over all items, it becomes:

$$\text{Minimize } \sum_{t=1}^n \sum_{u \in U_t} (r_{ut} - \sum_{j \in Q_t(u)} w_{jt}^{item} \cdot r_{uj})^2$$

- A regularization term should be included

Sparse Linear Models (SLIM)

- based on item-item regression
 - works with non-negative rating values.
1. does not restrict the regression coefficients to only the neighbourhood of the target item t .
 2. encourages sparsity in the regression coefficients with the use of regularization methods.

$$\left\{ \begin{array}{l} \hat{r}_{ut} = \sum_{j=1}^n w_{jt}^{item} \cdot r_{uj} \quad \forall u \in \{1 \dots m\}, \forall t \in \{1 \dots n\} \\ w_{tt}^{item} = 0 \\ w_{jt}^{item} \geq 0 \end{array} \right.$$

$$\begin{aligned}
\text{Minimize } J_t^s &= \sum_{u=1}^m (r_{ut} - \hat{r}_{ut})^2 + \lambda \cdot \sum_{j=1}^n (w_{jt}^{item})^2 + \lambda_1 \cdot \sum_{j=1}^n |w_{jt}^{item}| \\
&= \sum_{u=1}^m (r_{ut} - \sum_{j=1}^n w_{jt}^{item} \cdot r_{uj})^2 + \lambda \cdot \sum_{j=1}^n (w_{jt}^{item})^2 + \lambda_1 \cdot \sum_{j=1}^n |w_{jt}^{item}|
\end{aligned}$$

subject to:

$$w_{jt}^{item} \geq 0 \quad \forall j \in \{1 \dots n\}$$

$$w_{tt}^{item} = 0$$

- Note the elastic-net regularizer

*SLIM models work better than standard CF
It is one of the best approaches for Rec Sys.*

Sparse Linear Models (II)

Why SLIM works better than standard CF?

$$\hat{X} = X * W \circ \circ \circ$$

What will happen
if I try to learn W ???

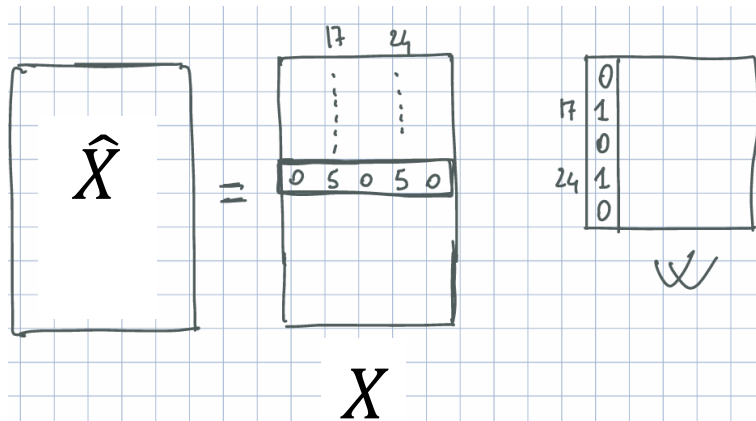
We apply these constraints:

$$\begin{cases} \text{diag}(W) = \bar{0} \\ w_{ij} \geq 0 \end{cases}$$

What does W become at the end of the learning ?

Let us rethink of collaborative filtering in terms of matrices

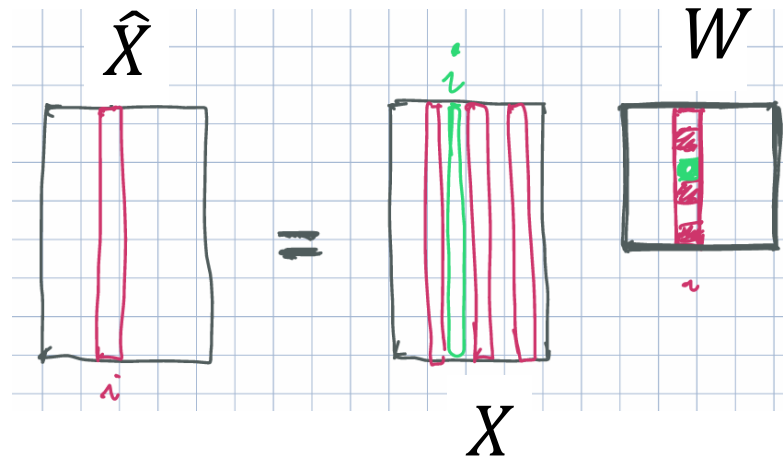
- If W were a similarity, the model would work



- Assume a high similarity between movies 17, 24, and 1
- Assume user liked movies 17 and 24
- Movie 1 will get a high score for the user

Learning W with SLIM, do I then get a similarity matrix?

$$\hat{X} = X * W$$



- $\hat{X}$ is a linear combination of the columns of X mixed by the coefficients in the columns of W
- To generate column i of $\hat{X}$ (that related to the i^{th} movie) I need to combine linearly different columns of X , and since it must be sparse, I will put together those that are most similar to the i^{th} movie.
- And note that I cannot use the i^{th} column directly because it is zero...

SO THIS IS CANNOT BE A SIMILARITY !!!